Backend Developer Technical Challenge — Wallet Service

Context

Imagine you are working on the backend of a digital finance application.

Users have wallets containing money, and the system needs to support transfers between those wallets. The service will eventually be used by multiple clients and may receive concurrent or repeated requests.

Your task is to design and implement a small backend service that handles these transfers reliably.

We are interested not only in whether the application works, but also in the engineering decisions you make along the way.

Your Task

Build a REST API using **Java or Kotlin and Spring Boot** that manages wallets and transfers between them.

You may choose the specific project structure, libraries, persistence technology, and implementation approach.

1. Wallets

A wallet should contain at least:

- a unique identifier
- an owner or display name
- a currency
- a current balance

The API should provide a way to:

- create a wallet
- retrieve a wallet
- retrieve its current balance

You may decide how the initial balance is created.

2. Transfers

Implement an endpoint for transferring money between two wallets.

A transfer should contain at least:

- a unique identifier
- source wallet
- destination wallet
- amount
- currency
- creation timestamp
- status

For example:

```
POST /transfers
```

```
{
  "sourceWalletId": "1b23...",
  "destinationWalletId": "8e91...",
  "amount": 25.50,
  "currency": "EUR"
}
```

After a successful transfer:

```
source balance      = source balance - amount
destination balance = destination balance + amount
```

Business Rules

At minimum, the service should enforce the following rules:

1. The transfer amount must be greater than zero.
2. The source and destination wallet must exist.
3. A wallet cannot transfer money to itself.
4. The source wallet must contain sufficient funds.
5. The currencies of both wallets and the transfer must match.
6. A transfer must either complete entirely or have no effect at all.

Return meaningful HTTP status codes and error responses when a request cannot be processed.

Reliability

Assume this service runs in a distributed environment.

Clients may retry requests when they do not receive a response.

Consider the following scenario:

```
Client → POST /transfers
Server → transfer succeeds
Network → response is lost
Client → retries POST /transfers
```

Your implementation should provide a reasonable mechanism for preventing the same logical transfer from being executed twice.

You may define the API contract required to support this.

Concurrency

Two transfer requests may attempt to spend money from the same wallet at approximately the same time.

For example:

```
Wallet balance: €100

Transfer A: €80
Transfer B: €70
```

The system must not end up allowing both transfers to succeed.

Choose an approach that keeps the wallet balance consistent and briefly explain your decision.

Persistence

Use persistent storage for the application's core data.

A relational database is a natural choice, but the specific technology is up to you.

Include whatever database schema, migrations, or initialization mechanism is needed to run the project.

If you believe a NoSQL database would be useful for part of the solution, feel free to include one, but this is not required.

Testing

Include automated tests for the parts of the system you consider most important.

We would expect to see a sensible combination of:

- unit tests
- integration tests

Pay particular attention to the business rules and transfer consistency.

You do not need to maximize test coverage. We are more interested in the quality and relevance of the tests.

Running the Application

The project should be easy for another developer to run.

Please provide a README containing:

- prerequisites
- instructions for starting the application
- instructions for running the tests
- a short description of the API

Docker and/or Docker Compose may be used if you find them useful.

Architecture Notes

In your README, include a short section called **Architecture & Decisions**.

Briefly explain:

- how you structured the application
- how transaction consistency is guaranteed
- how you handle concurrent transfers
- how duplicate requests are handled
- one or two trade-offs you deliberately made
- what you would change if the service had to handle significantly more traffic

Keep this section concise. A few paragraphs are enough.

Production Scenario

Finally, answer the following question in the README:

> A few weeks after deployment, monitoring shows that some transfer requests occasionally take 5–10 seconds instead of the usual 100 ms. How would you investigate the problem?

There is no single correct answer. Describe how you would approach the investigation and what information you would look for.

What We Evaluate

We will primarily look at:

Correctness
Does the implementation satisfy the business requirements and preserve data consistency?

Code quality
Is the code understandable, maintainable, and appropriately structured?

Backend design
Are the API, persistence model, transaction boundaries, and error handling thoughtfully designed?

Testing
Are important behaviors covered by useful automated tests?

Engineering judgment
Do the chosen solutions have reasonable complexity for the problem?

Production awareness
Does the solution consider concurrency, retries, failures, observability, and operational behavior?

Communication
Can another engineer understand your decisions and run your solution without unnecessary effort?

Optional Extensions

The following are entirely optional and should only be attempted if you have time:

- OpenAPI / Swagger documentation
- Docker Compose setup
- CI pipeline
- health/readiness endpoints
- structured logging
- metrics
- transfer history with pagination
- asynchronous transfer notifications
- a NoSQL-backed audit/event view

Optional features will not compensate for problems in the core implementation.

Submission

Please submit:

- the source code in a Git repository or ZIP archive
- the README
- all files necessary to run the application

Please do not spend excessive time polishing the solution. We would rather see a focused implementation with clear decisions and trade-offs than an unnecessarily large application.